

《LeetCode零基础指南》(第三讲) 一维数组

文章目录

- 零、了解网站
 - 1、输入输出
 - 2、刷题步骤
 - 3、尝试编码
 - 4、调试提交
- 一、概念定义
 - 1、顺序存储
 - 2、存储方式
 - 3、长度和容量
 - 4、数组的索引
 - 5、数组的函数传参
- 二、题目分析
 - 1、数组的查找
 - 2、数组的最小值
 - 3、斐波那契数列
 - 4、绝对值为 k 的数对
 - 5、猜数字
 - 6、拿硬币
- 三、推荐学习
- 四、课后习题

零、了解网站

1、输入输出

对于算法而言，就是给定一些输入，得到一些输出，在一般的在线评测系统中，我们需要自己手写输入输出函数（例如 C 语言中的 `scanf` 和 `printf`），而在 `LeetCode` 这个平台，只需要实现它提供的函数即可。函数的传入参数就代表了算法的输入，而函数的返回值则代表了算法的输出。

2、刷题步骤

找到一道题，以 `两整数之和` 为例，如何把这道题通过呢？如下图所示：

力扣 学习 题库 讨论 竞赛 求职 商店

新功能 Plus 会员 我是面试官

题目描述 评论 (548) 题解 (510) 提交记录

371. 两整数之和

难度 中等 554

给你两个整数 a 和 b ，不使用运算符 $+$ 和 $-$ ，计算并返回两整数之和。

示例 1:

输入: $a = 1, b = 2$
输出: 3

示例 2:

输入: $a = 2, b = 3$
输出: 5

提示:

- $-1000 \leq a, b \leq 1000$

1 2 3 4 5 6

```
1 int getSum(int a, int b){
2
3
4 }
```

执行代码 提交

- 第 1 步: 阅读题目;
- 第 2 步: 参考示例;
- 第 3 步: 思考数据范围;
- 第 4 步: 根据题意, 实现函数的功能;
- 第 5 步: 本地数据测试;
- 第 6 步: 提交;

3、尝试编码

这个题目比较简单, 就是求两个数的和, 我们可以在编码区域尝试敲下这么一段代码。

```
1 int getSum(int a, int b){ // (1)
2     return a + b;        // (2)
3 }
```

- (1) 这里 `int` 是 C/C++ 中的一种类型, 代表整数, 即 Integer, 传入参数是两个整数;
- (2) 题目要求返回两个整数的和, 我们用加法运算符实现两个整数的加法;

4、调试提交

力扣

学习

题库

讨论

竞赛

求职

商店

新功能

Plus 会员

我是面试官

中

题目描述

评论 (548)

题解 (510)

提交记录

执行结果: 通过 显示详情 >

添加

执行用时: 0 ms, 在所有 C 提交中击败了 100.00% 的用户

内存消耗: 5.5 MB, 在所有 C 提交中击败了 17.94% 的用户

通过测试用例: 25 / 25

炫耀一下:

写题解, 分享我的解题思路

提交结果

执行用时

内存消耗

提交时间

备注

通过

通过

0 ms

5.5 MB

C

2021/11/22 22:33

添加

0 ms

5.2 MB

C

2021/11/22 13:21

添加

题目...

随机

上一题

371/2436

下一题

C

智能模式

模拟面试

i

1

2

3

4

int getSum(int a, int b){

return a + b;

}

测试用例

代码执行结果

调试器 Beta

已完成

执行用时: 0 ms

输入

45

289

输出

334

3

2

5

差别

预期结果

334

4

控制台

填入示例

如何创建一个测试用例?

执行代码

提交

- 第 1 步: 实现加法, 将值返回;
- 第 2 步: 执行代码, 进行测试数据的调试;
- 第 3 步: 代码的实际输出;
- 第 4 步: 期望的输出, 如果两者符合, 则代表这一组数据通过 (但是不代表所有数据都能通过哦);
- 第 5 步: 尝试提交代码;
- 第 6 步: 提交通过, 撒花 🌸🌸🌸🌸🌸🌸;

这样, 我们就大致知道了一个刷题的步骤了。上一章我们了解了循环, 今天我们就来学习一下数组, 从 **一维数组** 开始学习。

一、概念定义

1、顺序存储

顺序存储结构, 是指用一段地址连续的存储单元来依次存储数据。如图所示, 每个蓝色方块都对应对应了数组中的一个数据。数据有类型, 例如: 32位整型 `int`、单精度浮点型 `float`、双精度浮点型 `double`、字符型 `char`, 64位整型 `long long`、16位短整型 `short` 等等。



2、存储方式

在编程语言中, 用一维数组来实现顺序存储结构, 在C语言中, 把第一个数据元素存储到下标为 0 的位置中, 把第 2 个数据元素存储到下标为 1 的位置中, 以此类推。

C语言中, 数组的定义如下:

```
1 | int a[7];
```

我们也可以将数组元素进行初始化, 如下:

```
1 | int a[7] = {5, 2, 0, 1, 3, 1, 4};
```

0	1	2	3	4	5	6
5	2	0	1	3	1	4

由于编译器能够通过后面的初始化列表知道到底有多少个数据元素, 所以中括号中的数字可以省略, 如下:

```
1 | int a[] = {5, 2, 0, 1, 3, 1, 4};
```

当然，我们还可以定义一个大数组，但是只初始化其中几个元素：

```
1 | int a[18] = {5, 2, 0, 1, 3, 1, 4};
```

3、长度和容量

数组的长度指的是数组当前有多少个元素，数组的容量指的是数组最大能够存放多少个元素。如果数组元素大于最大能存储的范围，在程序上是不允许的，可能会产生意想不到的问题，实现上是需要规避的。



如上图所示，数组的长度为 5，即红色部分；容量为 8，即红色 加 蓝色部分。例如下面这个数组：

```
1 | int a[8] = {9, 8, 7, 6, 5};
```

4、数组的索引

数组中的元素索引，我们可以采用 `[]` 运算符来完成，例如：

```
1 | int a[7] = {5, 2, 0, 1, 3, 1, 4};
2 | int b = a[0];    // 5
3 | int c = a[6];    // 4
4 | int d = a[7];    // error
```

这段代码中，`a[0]` 表示的是数组的第一个元素，`a[6]` 表示的数组的最后一个元素。

而 `a[7]` 则是非法的，原因是数组元素一个 7 个，而这个调用，索引到了第 8 个元素，如果程序这样调用，就可能引起位置的错误。这种错误调用，我们一般称它为：数组下标越界。

5、数组的函数传参

在学习函数的时候，我们知道，如果想将两个整型变量传递给函数，通过函数返回两个整型变量的和，我们可以写出如下代码：

```
1 | int add(int a, int b) {
2 |     return a + b;
3 | }
```

那么，如果我们要求的是一个长度为 10 的数组中所有元素的和，我们如何通过函数来实现呢？我们学过循环 和 函数，并且套用上面的格式，传递参数变成一个数组，如下：

```
1 | int add(int nums[10]) {
2 |     int i;
3 |     int s = 0;
4 |     for(i = 0; i < 10; ++i) {
5 |         s += nums[i];
6 |     }
7 |     return s;
8 | }
```

我们利用一个循环，把数组中所有的元素取出来，然后累加到变量 `s` 上，最后返回 `s` 的值，那么问题来了，如果没有定义数组的长度，我们如何完成这件事情呢？

对于计算机来说，它其实并不知道你的数组有多少个元素，因为这是你自己定义的，所以你需要告诉这个函数，所以，我们只需要在传参中再添加一个参数，代表数组的长度即可，代码实现如下：

```
1 | int add(int nums[10], int numsSize) {
2 |     int i;
3 |     int s = 0;
4 |     for(i = 0; i < numsSize; ++i) {
5 |         s += nums[i];
6 |     }
7 |     return s;
8 | }
```

以上代码中，我们用 `numsSize` 来代表这个数组中有多少个元素，然后通过参数的形式传递给这个函数，那么，函数在计算的时候就可以取变量 `numsSize` 的值作为循环上限来进行累加了。

并且，我们还可以把参数列表中的数字 10 去掉，如下：

```
1 | int add(int nums[], int numsSize) {
2 |     // ...
3 | }
```

在力扣这个平台，传参一般会写成如下形式：

```
1 int add(int *nums, int numsSize) {
2     // ...
3 }
```

这里的 `int *nums` 和 `int nums[]` 是等价的，其中 `*` 表示指针，好了，指针是下一章的内容，今天数组相关的内容就介绍到这里了，你只需要理解两者是等价的，今天的题就可以做了。

二、题目分析

1、数组的查找

实现一个函数，在一个整型数组中找一个整数，返回它的数组下标，如果不存在则返回 `-1`。

```
1 int search(int* nums, int numsSize, int target){ // (1)
2     int i;
3     for(i = 0; i < numsSize; ++i) {           // (2)
4         if(nums[i] == target) {
5             return i;                         // (3)
6         }
7     }
8     return -1;                                // (4)
9 }
```

- (1) 传入参数 `int * nums` 代表一个整型数组，`int numsSize` 代表数组长度，`target` 则表示要查找的数；
- (2) 遍历数组的每个元素；
- (3) 如果找到某个元素和传入参数 `target` 相等，则直接返回下标 `i`；
- (4) 如果一直没找到，则返回 `-1`；

2、数组的最小值

实现一个函数，在一个整型数组中寻找最小值并返回，数组元素值都不超过 100000。

```
1 int findMin(int* nums, int numsSize){
2     int i, min = 100000;           // (1)
3     for(i = 0; i < numsSize; ++i) { // (2)
4         if(nums[i] < min) {         // (3)
5             min = nums[i];
6         }
7     }
8     return min;
9 }
```

- (1) 定义一个全局最小值；
- (2) 遍历数组所有元素；
- (3) 如果数组元素 `nums[i]` 比全局最小值小，则更新全局最小值。

3、斐波那契数列

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

```
1 int f[1000];           // (1)
2 int climbStairs(int n){
3     f[0] = f[1] = 1;    // (2)
4     for(int i = 2; i <= n; ++i) {
5         f[i] = f[i-1] + f[i-2]; // (3)
6     }
7     return f[n];        // (4)
8 }
```

- (1) 定义一个全局的辅助数组；
- (2) 初始化方案为 1；
- (3) 你可以从 $i - 1$ 层爬过来，也可以从 $i - 2$ 层爬过来，所以两个方案数加和即可；
- (4) 返回到达第 n 层的方案数；

4、绝对值为 k 的数对

给你一个整数数组 `nums` 和一个整数 `k`，请你返回数对 (i, j) 的数目，满足 $i < j$ 且 $|nums[i] - nums[j]| == k$ 。

```
1 int countKDifference(int* nums, int numsSize, int k){
2     int i, j, ans = 0;
3     for(i = 0; i < numsSize; ++i) {           // (1)
4         for(j = i + 1; j < numsSize; ++j) {    // (2)
5             if( abs(nums[i] - nums[j]) == k )  // (3)
6                 ++ans;
7         }
8     }
9     return ans;                               // (4)
10 }
```

- (1) 先枚举一个 i ;
- (2) 再枚举一个 j ;
- (3) 然后计算两个数组元素的差的绝对值，判断是否等于 k ，如果等于，计数器加一；
- (4) 最后返回计数器；

5、猜数字

小A 和 小B 在玩猜数字。小B 每次从 1, 2, 3 中随机选择一个，小A 每次也从 1, 2, 3 中选择一个猜。他们一共进行三次这个游戏，请返回 小A 猜对了几次？输入的 `guess` 数组为 小A 每次的猜测，`answer` 数组为 小B 每次的选择。guess和answer的长度都等于3。

```
1 int game(int* guess, int guessSize, int* answer, int answerSize){
2     int i;
3     int ans = 0;
4     for(i = 0; i < 3; ++i) {
5         ans += (guess[i] == answer[i]) ? 1 : 0;  // (1)
6     }
7     return ans;
8 }
```

- (1) 分别取两个数组中对应的元素进行判断，如果相等则计数器加一，否则加零（即不加）；

6、拿硬币

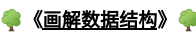
桌上有 n 堆力扣币，每堆的数量保存在数组 `coins` 中。我们每次可以选择任意一堆，拿走其中的一枚或者两枚，求拿完所有力扣币的最少次数。

```
1 int minCount(int* coins, int coinsSize){
2     int i;
3     int ans = 0;
4     for(i = 0; i < coinsSize; ++i) {
5         ans += (coins[i]+1)/2;  // (1)
6     }
7     return ans;
8 }
```

- (1) 这个问题其实就是奇数 和 偶数 的情况分别处理即可，考虑一个数 x ，如果是奇数，就要返回 $\frac{x+1}{2}$ ；如果是偶数，则返回 $\frac{x}{2}$ 。当然也可以合并，利用 C语言整数除法是取下整的性质，可以都用

$$\frac{x + 1}{2}$$

三、推荐学习



《画解数据结构》

(1-1) 顺序表

四、课后习题

课后所有习题，在上文都能找到答案，所以务必将 [必做] 为 1 的题全部刷完。第四天了，再坚持一天TM就是旅程的一半了，你好意思放弃！你这个年龄，好意思放弃！？
坚持！加油！你可以的！

序号	题目链接	难度	必做
----	------	----	----

序号	题目链接	难度	必做
1	搜索旋转排序数组	★☆☆☆☆	1
2	搜索旋转排序数组 II	★☆☆☆☆	1
3	寻找旋转排序数组中的最小值	★☆☆☆☆	1
4	爬楼梯	★☆☆☆☆	1
5	斐波那契数	★☆☆☆☆	1
6	第 N 个泰波那契数	★☆☆☆☆	1
7	差的绝对值为 K 的数对数目	★☆☆☆☆	0
8	猜数字	★☆☆☆☆	0
9	拿硬币	★☆☆☆☆	0
10	山峰数组的顶部	★☆☆☆☆	0

👉 关注公众号 观看 精彩学习视频 👈